

# COMPTE RENDU

*Projet : Système de Gestion des Médicaments*

|              |                                   |
|--------------|-----------------------------------|
| Langages     | Python 3.10+, SQL (PostgreSQL)    |
| Modélisation | UML, Graphe de connaissances, ERD |
| Etudiantes   | AMZAOUROU Nada<br>ZEBDI Sara      |

# 1. Introduction

---

Dans le cadre de ce projet, nous avons conçu et développé un système complet de gestion des médicaments destiné à une pharmacie ou un centre médical. Ce système couvre l'ensemble du cycle de vie d'un médicament : de son référencement initial jusqu'à sa dispensation au patient, en passant par la gestion du stock, la création d'ordonnances et leur validation par le pharmacien.

L'objectif principal est de modéliser et d'implémenter un système fiable permettant :

- la gestion complète des médicaments (nom, dosage, forme, date d'expiration, prix),
- le suivi du stock avec alertes de rupture et historique des mouvements,
- la gestion des patients et de leurs allergies,
- la création et la validation des ordonnances médicales,
- la dispensation sécurisée des médicaments par les pharmaciens.

Le projet a été réalisé en combinant trois technologies complémentaires :

- la programmation orientée objet (POO) en Python,
- une base de données relationnelle SQL (PostgreSQL),
- la modélisation UML avec diagramme de classes et graphe de connaissances.

## 2. Analyse des besoins

---

Avant toute implémentation, nous avons identifié les acteurs du système et les fonctionnalités attendues.

### 2.1 Acteurs du système

| Acteur     | Rôle                                                                    |
|------------|-------------------------------------------------------------------------|
| Patient    | Reçoit des ordonnances, peut avoir des allergies déclarées              |
| Pharmacien | Valide les ordonnances, vérifie les allergies, dispense les médicaments |
| Médicament | Entité centrale : référencé, stocké, prescrit et dispensé               |
| Ordonnance | Document liant un patient à un ensemble de médicaments prescrits        |

### 2.2 Fonctionnalités requises

- Ajouter, consulter et supprimer des médicaments.
- Gérer les stocks : quantité disponible, seuil d'alerte, réapprovisionnement.
- Enregistrer les patients et leurs allergies médicamenteuses.
- Créer des ordonnances avec plusieurs lignes (médicament, posologie, durée).
- Valider une ordonnance : vérifier la disponibilité du stock et les allergies.
- Dispenser les médicaments et mettre à jour le stock automatiquement.
- Calculer le coût total d'une ordonnance.
- Conserver un historique des mouvements de stock.

## 3. Conception du système

### 3.1 Diagramme de classes UML

Le diagramme de classes UML est la pierre angulaire de la conception du système. Il représente les six classes principales et les relations qui les unissent.

Le diagramme de classes ci-dessous modélise le système selon trois types de relations : la composition (Stock est indissociable de Medicament), l'agrégation (LigneOrdonnance regroupe des Medicament dans une Ordonnance) et l'association (Patient, Pharmacien ↔ Ordonnance).

| Classe          | Attributs principaux                          | Méthodes clés                                          |
|-----------------|-----------------------------------------------|--------------------------------------------------------|
| Medicament      | id, nom, dosage, forme, date_expiration, prix | est_expire()                                           |
| Stock           | quantite, seuil_alerte                        | verifier_stock(), reapprovisionner(), est_disponible() |
| Patient         | id, nom, prenom, date_naissance, allergies    | get_historique(), age (property)                       |
| Pharmacien      | id, nom, prenom, numero_licence               | valider_ordonnance(), dispenser_medicament()           |
| Ordonnance      | id, date_emission, date_validite, statut      | ajouter_medicament(), est_valide(), calculer_total()   |
| LigneOrdonnance | quantite, posologie, duree_traitement         | calculer_sous_total()                                  |

### 3.2 Relations entre les classes

Les relations entre les classes du diagramme UML reflètent les règles métier de la pharmacie :

- Composition (1-1) : chaque Medicament possède exactement un Stock. Le stock ne peut exister sans son médicament.
- Association (1-N) : un Patient peut avoir plusieurs Ordonnances ; un Pharmacien peut valider plusieurs Ordonnances.
- Agrégation (1-N) : une Ordonnance contient une ou plusieurs LigneOrdonnance.
- Référence : chaque LigneOrdonnance pointe vers un Medicament existant.

### 3.3 Graphe de connaissances

Le graphe de connaissances complète le diagramme UML en représentant les relations sémantiques entre entités sous forme de nœuds et d'arêtes orientées. Contrairement au diagramme de classes, le graphe ne montre pas les attributs mais la signification métier des liens.

Le graphe comporte 6 nœuds (entités) et 8 relations. Les arêtes pleines représentent des associations directes (composition, agrégation) ; les arêtes en pointillé représentent des relations dérivées ou navigables dans les deux sens.

| Source          | Cible           | Relation sémantique          |
|-----------------|-----------------|------------------------------|
| Medicament      | Stock           | a un stock (composition 1-1) |
| Medicament      | LigneOrdonnance | référéncé dans               |
| LigneOrdonnance | Ordonnance      | appartient à                 |
| Ordonnance      | Patient         | prescrite à                  |
| Ordonnance      | Pharmacien      | validée par                  |
| Patient         | Ordonnance      | possède (relation inverse)   |
| Pharmacien      | Medicament      | dispense                     |
| Ordonnance      | LigneOrdonnance | contient (1..*)              |

## 4. Implémentation en Python

### 4.1 Structure du code

Le code Python suit rigoureusement le diagramme UML. Chaque classe est implémentée avec ses attributs, son constructeur et ses méthodes. Les classes utilisent des compteurs statiques (`_compteur`) pour générer des identifiants auto-incrémentés, à l'image d'une base de données.

### 4.2 Principe d'héritage en POO

L'héritage est l'un des piliers de la programmation orientée objet. Il permet à une classe fille de hériter des attributs et des méthodes d'une classe mère, évitant ainsi la duplication de code et favorisant la réutilisabilité.

Principe : la classe fille (enfant) étend la classe mère (parent). Elle dispose automatiquement de tout ce que possède la mère, et peut y ajouter ses propres attributs et méthodes, ou redéfinir (override) ceux hérités.

#### 4.2.1 Application à notre système

Dans notre projet, l'héritage peut être appliqué pour factoriser les attributs communs à toutes les personnes impliquées dans le système (Patient et Pharmacien partagent nom, prenom, id). On crée une classe mère `Personne` dont héritent les deux :

```
class Personne:
    _compteur = 0

    def __init__(self, nom: str, prenom: str):
        Personne._compteur += 1
        self.id = Personne._compteur
```

```

        self.nom      = nom
        self.prenom = prenom

    def __repr__(self):
        return f'Personne({self.prenom} {self.nom})'

class Patient(Personne):
    # hérite de Personne
    def __init__(self, nom, prenom, date_naissance, allergies=None):
        super().__init__(nom, prenom) # appel du constructeur parent
        self.date_naissance = date_naissance
        self.allergies       = allergies or []
        self._historique     = []

    def get_historique(self):
        return list(self._historique)

class Pharmacien(Personne):
    # hérite aussi de Personne
    def __init__(self, nom, prenom, numero_licence):
        super().__init__(nom, prenom) # appel du constructeur parent
        self.numero_licence = numero_licence

    def valider_oronnance(self, ordonnance):
        # ... logique de validation
        pass

```

#### 4.2.2 Avantages de l'héritage

| Avantage        | Explication                                                                   |
|-----------------|-------------------------------------------------------------------------------|
| Réutilisabilité | Les attributs id, nom, prenom sont définis une seule fois dans Personne       |
| Extensibilité   | Ajouter un Médecin, Infirmier... suffit de créer une classe fille de Personne |
| Maintenabilité  | Modifier un attribut commun se fait à un seul endroit                         |
| Polymorphisme   | Une liste de Personne peut contenir patients et pharmaciens indifféremment    |
| super()         | Permet d'appeler le constructeur ou une méthode de la classe mère             |

#### 4.2.3 Surcharge de méthode (Override)

Chaque classe fille peut redéfinir la méthode `__repr__` héritée de Personne pour afficher des informations spécifiques :

```

class Personne:
    def __repr__(self):
        return f'Personne({self.prenom} {self.nom})'

class Patient(Personne):
    def __repr__(self):
        # override de Personne.__repr__
        return f'Patient({self.prenom} {self.nom}, {self.age} ans)'

class Pharmacien(Personne):

```

```
def __repr__(self):
    # override de Personne.__repr__
    return f'Pharmacien({self.prenom} {self.nom},
licence={self.numero_licence!r})'
```

## 4.3 Fonctionnalités clés implémentées

### Gestion du stock

- `verifier_stock()` retourne l'état : OK / ALERTE / RUPTURE.
- `reapprovisionner(n)` incrémente la quantité et journalise l'opération.
- `est_disponible(n)` vérifie à la fois la quantité et la date d'expiration.

### Validation d'une ordonnance

- Vérification de la date de validité de l'ordonnance.
- Vérification du stock disponible pour chaque ligne.
- Contrôle des allergies du patient par rapport aux médicaments prescrits.
- Mise à jour du statut : en attente → validée → dispensée.

### Calcul du coût

La méthode `calculer_total()` d'`Ordonnance` itère sur toutes les `LigneOrdonnance` et somme les sous-totaux ( $\text{quantite} \times \text{prix\_unitaire}$ ), offrant ainsi un total précis de la dispensation.

## 5. Base de données SQL

### 5.1 Schéma relationnel (ERD)

La base de données reflète fidèlement le modèle UML. Chaque classe devient une table, chaque relation devient une clé étrangère (FOREIGN KEY). Le schéma ERD (Entity-Relationship Diagram) a été modélisé avec la notation crow's foot.

| Table            | Colonnes principales                                      | Clés étrangères              |
|------------------|-----------------------------------------------------------|------------------------------|
| pharmacien       | id, nom, prenom, numero_licence                           | —                            |
| patient          | id, nom, prenom, date_naissance, numero_secu              | —                            |
| allergie         | id, substance, severite                                   | patient_id → patient         |
| medicament       | id, nom, dosage_mg, forme, date_expiration, prix_unitaire | —                            |
| stock            | id, quantite, seuil_alerte, emplacement                   | medicament_id → médicament   |
| mouvement_stock  | id, type, quantite, raison                                | stock_id, pharmacien_id      |
| ordonnance       | id, date_emission, date_validite, statut                  | patient_id, pharmacien_id    |
| ligne_ordonnance | id, quantite, posologie, duree_traitement                 | ordonnance_id, medicament_id |

## 5.2 Contraintes d'intégrité

- CHECK sur le statut de l'ordonnance : seules les valeurs en\_attente, validée, dispensée, annulée, expirée sont acceptées.
- CHECK sur les quantités : toujours > 0 pour les lignes, ≥ 0 pour le stock.
- UNIQUE sur numero\_licence (pharmacien) et numero\_secu (patient).
- Contrainte de date : date\_validite ≥ date\_emission sur chaque ordonnance.

## 5.3 Vues SQL créées

Trois vues ont été créées pour faciliter l'exploitation des données :

| Vue                      | Description                                                                       |
|--------------------------|-----------------------------------------------------------------------------------|
| v_stock_alerte           | Liste tous les médicaments dont la quantité est ≤ au seuil d'alerte ou en rupture |
| v_medicaments_expiration | Médicaments expirant dans les 90 prochains jours avec jours restants              |
| v_oronnances_actives     | Ordonnances en cours avec total en euros et nombre de médicaments                 |

## 5.4 Index de performance

Des index ont été créés sur les colonnes les plus fréquemment interrogées pour optimiser les performances :

- idx\_patient\_nom : recherche rapide par nom/prénom.
- idx\_medicament\_exp : contrôle quotidien des péremptions.
- idx\_oronnance\_statut : filtrage par état d'avancement.
- idx\_stock\_seuil : détection des ruptures en temps réel.

## 6. Tests réalisés

Après le développement du système, une série de tests fonctionnels a été réalisée pour valider l'ensemble des fonctionnalités :

| # | Scénario de test                                 | Résultat attendu                      | Statut |
|---|--------------------------------------------------|---------------------------------------|--------|
| 1 | Ajout d'un médicament et initialisation du stock | Stock créé avec quantite=0            | OK     |
| 2 | Réapprovisionnement de +200 unités               | Quantité = 200, alerte désactivée     | OK     |
| 3 | Vérification du stock sous seuil d'alerte        | Retourne ALERTE ou RUPTURE            | OK     |
| 4 | Création d'une ordonnance avec 2 lignes          | Ordonnance créée, statut = en_attente | OK     |
| 5 | Validation avec allergie déclarée                | Rejet de la validation + message      | OK     |

| # | Scénario de test                           | Résultat attendu                     | Statut |
|---|--------------------------------------------|--------------------------------------|--------|
| 6 | Validation sans allergie + stock suffisant | Statut passe à validée               | OK     |
| 7 | Dispensation des médicaments               | Stock décrémenté, statut = dispensée | OK     |
| 8 | Calcul du total d'une ordonnance           | Somme correcte des sous-totaux       | OK     |

## 7. Difficultés rencontrées

- Modélisation UML des relations (composition vs agrégation vs association) : résolu par l'analyse des dépendances de cycle de vie entre objets.
- Gestion du compteur statique `Personne._compteur` partagé entre `Patient` et `Pharmacien` après introduction de l'héritage : résolu en dédiant un compteur par classe fille.
- Organisation des clés étrangères en SQL pour respecter l'ordre de création des tables (les tables référencées doivent exister avant les tables référençant) : résolu par un ordonnancement explicite.
- Liaison cohérente entre le modèle Python et la base SQL : les noms, types et contraintes ont été alignés manuellement entre les deux couches.

## 8. Conclusion

Ce projet de gestion des médicaments illustre la complémentarité de trois disciplines fondamentales du génie logiciel : la modélisation, la programmation orientée objet et les bases de données relationnelles.

Sur le plan de la modélisation, le diagramme de classes UML nous a permis de structurer le domaine métier avant d'écrire la moindre ligne de code. Le graphe de connaissances a ensuite enrichi cette vision en révélant les relations sémantiques entre entités, tandis que le diagramme ERD a fourni le plan précis pour la construction de la base SQL.

Sur le plan du code Python, l'application du principe d'héritage — en particulier la hiérarchie `Personne` → `Patient` / `Pharmacien` — a démontré concrètement les bénéfices de la POO : élimination de la duplication, facilité d'extension et lisibilité accrue. L'utilisation de `super()` pour déléguer au constructeur parent, et la surcharge de `__repr__` pour adapter l'affichage à chaque classe, illustrent deux mécanismes essentiels de l'héritage en Python.

Sur le plan de la base de données, les huit tables SQL, leurs contraintes d'intégrité (CHECK, UNIQUE, FOREIGN KEY), leurs index de performance et les trois vues métier constituent un schéma robuste, directement exploitable en production avec PostgreSQL.

En définitive, ce projet démontre qu'une conception rigoureuse en amont — diagrammes UML, graphe de connaissances, schéma ERD — aboutit à une implémentation plus cohérente, plus maintenable et plus facilement extensible. Il constitue une base solide sur laquelle des fonctionnalités avancées pourraient être ajoutées : interface web, API REST, système de



notifications pour les péremptions imminentes, ou encore module d'intelligence artificielle pour la détection d'interactions médicamenteuses.

Le code source Python (`gestion_medicaments.py`) et le script SQL (`gestion_medicaments.sql`) sont fournis en annexe et constituent des livrables directement exécutables.